ARM - Advanced RISC Machines

RISC- Reduce Instruction Set Computers

ARM Design Philosophy

- ARM Core uses a RISC architecture
- ARM licenses its cores out and other companies make processors based on its cores
- ARM processor core based microcontrollers
- Von Neumann architecture.

ARM Design Philosophy

- Reduce power consumption
- High code density
- Price sensitive
- Reduce the area of the die taken up by the embedded processor
- Incorporated hardware debug technology (ICE)

ARM processor core based microcontrollers

- Key component of many 32 –bit embedded systems and Portable Consumer devices
- ARM1 prototype in 1985
- One of the ARM’s most successful cores is the ARM7TDMI,provides high code density and low power consumption.

ARM7TDMI

- T –***T***humb 16 bit compressed instruction set
- D – on chip ***D***ebug request
- M – enhanced ***M***ultiplier(yields 64 bit result)
- I – Embedded ***I***CE hardware to give on-chip breakpoint and watchpoint support

ICE – in circuit emulator for debugging

The RISC Design Philosophy

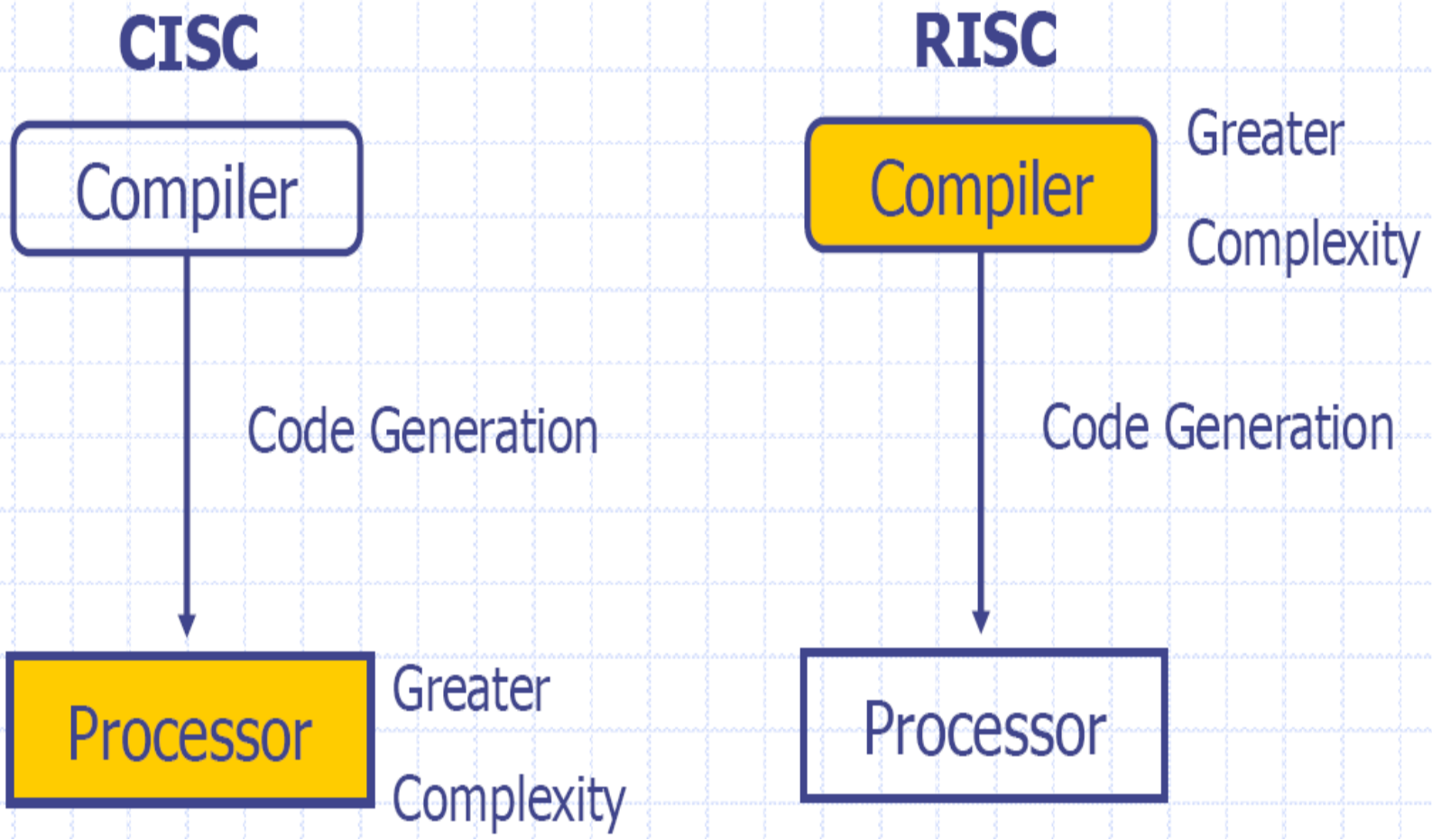
- RISC is characterized by limited number of instructions
- A complex instruction is obtained as a sequence of simple instructions.
- So, in RISC processor, software is complex but the processor architecture is simple.
- Large number of registers are required.
- Pipelined instruction execution.

- Ex : ARM, ATMEL AVR, MIPS, Power PC etc

The CISC Design Philosophy

- CISC is characterized by large instruction set.
- The aim of designing CISC processors is to reduce software complexity by increasing the complexity of processor architecture.
- Very small number of registers are available.
- Ex : Intel X86 family, Motorola 68000 series.

CISC vs. RISC


CISC
Compiler
Code Generation
Processor
Greater
Complexity
RISC
Compiler
Greater
Complexity
Code Generation
Processor

RISC –4 major design rules

1. Instructions

- Reduced Number of Instructions
- Execute in a single cycle
- The compiler synthesizes complicated operations
- Each instruction is a fixed length

2. Pipelines

- The processing of instructions is broken down into smaller units that can be executed in parallel by pipelines
- Pipeline advances by one step on each cycle for maximum throughput

3. Registers

- Have a large general purpose register set
- Any register can contain either data or address
- CISC has dedicated registers for specific purposes.

4. Load –Store Architecture

- Separate load and store instructions transfers data between the register bank and external memory.
- Memory accesses are costly, so separating memory access from data processing provides an advantage, because you can use data items held in register banks multiple times without needing multiple memory accesses.

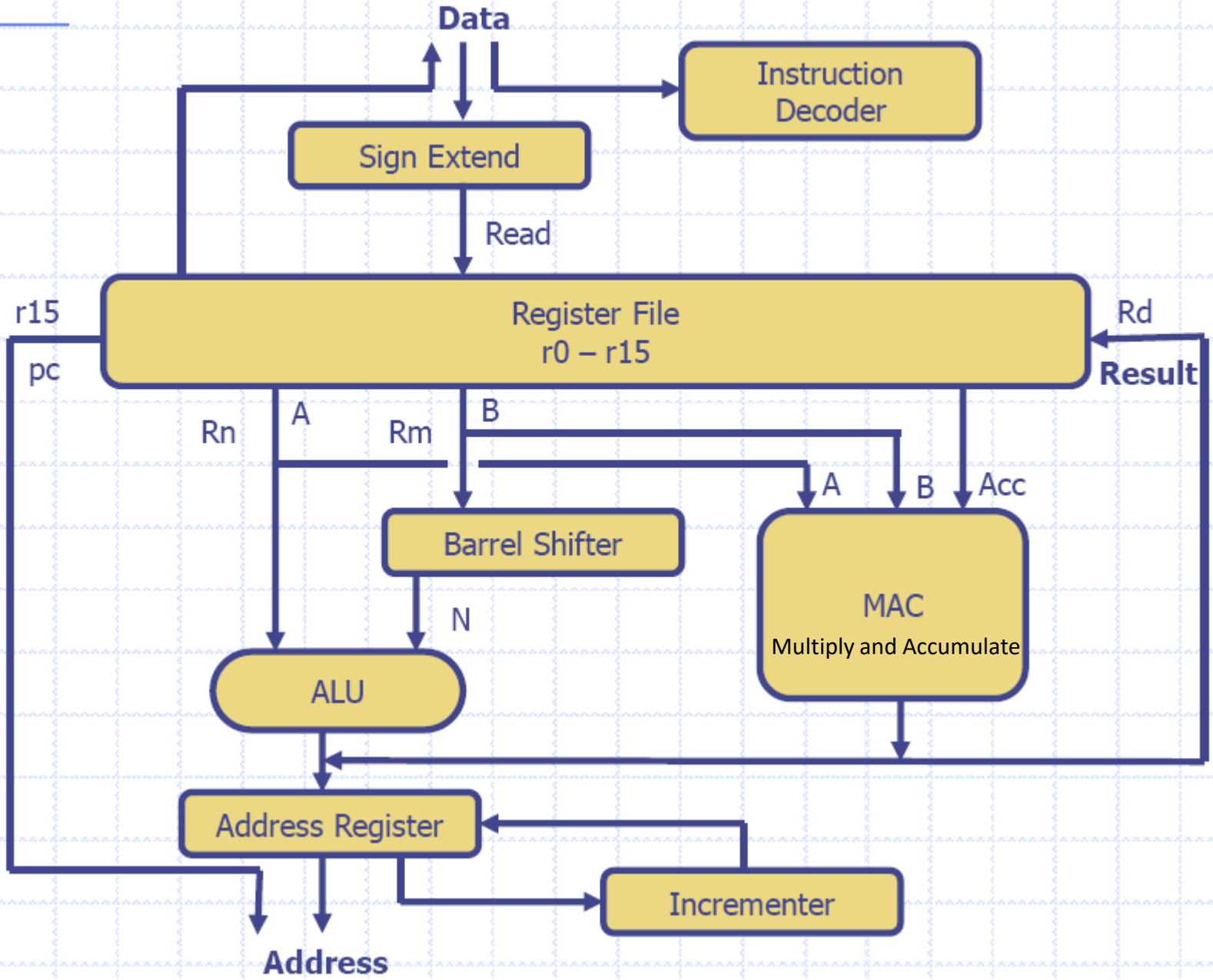
Load/Store Architecture

- Memory accesses slow a processor down.
- There are times when the processor is doing nothing, while waiting for memory accesses to complete.
- So, we define a new architecture. In this new ***load/store architecture***, the addressing mode for every operand is fixed. (So, there are no bytes for addressing mode information.)
- And, for arithmetic/logical type instructions, the addressing mode for all operands will be register mode.
- We make sure that there are enough registers, because everything ends up in registers.
- To get stuff to/from memory and into/out of registers, we have explicit instructions that move data.
- Load instructions read data from memory and copy it to a register. Store instructions write data from a register to memory.

Overview: Core Data Path

- Data items are placed in register file
  - No data processing instructions directly manipulate data in memory
- Instructions typically use two source registers and single result or destination register
- A Barrel shifter on the data path can pre-process data before it enters ALU
- Increment/Decrement logic can update register content for sequential access independent of ALU

ARM core dataflow model



ARM core dataflow model

- Functional units connected by data buses
- **Data Bus** → Data or Instruction
- Von Neumann architecture → Data and instructi
share the same bus
- Load Store Architecture
- Register File – 32 bit registers
- ARM instruction has 2 source and 1 destination register
- L/S inst: use the ALU to generate an address to held in the address reg. and broadcast on the **Address Bus**
- **Result Bus**

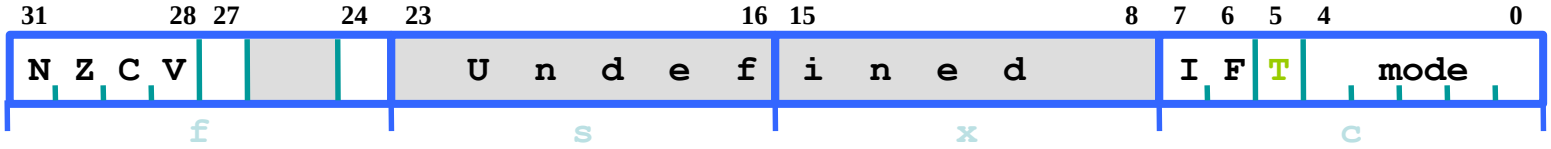
Registers

- General purpose registers hold either data or address
- All registers are of 32 bits
- In user mode 16 data registers and 2 status registers are visible
- Data registers: r0 to r15
  - Three registers r13, r14 and r15 perform special functions
  - r13: stack pointer
  - r14: link register (where return address is stored whenever a subroutine is called)
  - r15: program counter

Registers contd..

- Depending upon context r13 and r14 can also be used as GPR
- Any instruction which use r0 can as well be used with any other GPR(r1-r13)
- In addition, there are two status registers
  - CPSR: Current Program Status Register
  - SPSR: Saved Program Status Register

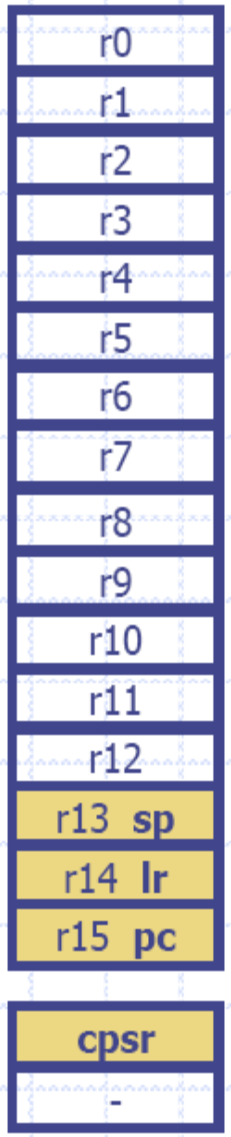
Program Status Registers



- Condition code flags
  - N = **N**egative result from ALU
  - Z = **Z**ero result from ALU
  - C = ALU operation **C**arried out
  - V = ALU operation o**V**erflowed
- Interrupt Disable bits.
  - I = 1: Disables the IRQ.
  - F = 1: Disables the FIQ.
- T Bit
  - Architecture xT only
  - T = 0: Processor in ARM state
  - T = 1: Processor in Thumb state
- Mode bits
  - Specify the processor mode

Registers – User Mode

- 32 bit in size
- Hold either data or address
- 16 data registers(r0 – r15) and 2 processor status register (cpsr & spsr)
- r13, r14, r15 – Special functions
- r13 (sp) –stores the head of the stack in the current processor mode
- r14 (lr) – the core puts the return address whenever it calls a subroutine
- r15 (pc) – contains the address of the next instruction to be fetched by the processor
- Which register are visible to the programmer depend upon the current mode of the processor



Processor Modes

- Abort – Failed attempt to access memory
- FIQ IRQ – Two interrupt levels available o the ARM processor
- Supervisor – OS kernel operates
- System – Special version of user mode th allows full R/W access to the cpsr
- Undefined – processor encounters an instruction that is undefined
- User – used in programs & applications
- When a power is applied to the core it sta in supervisor mode.

Processor Modes

- Determines which registers are active and the access rights to the cpsr register itself
- Privileged & Nonprivileged
  - Abort
  - Fast Interrupt Request
  - Interrupt Request
  - Supervisor
  - System
  - Undefined
  - User

Privileged-R/W access to CPSR

Nonprivileged-R Access to CF,R/W access to ConditionFlags

Processor modes

- Processor modes determine
  - Which registers are active and
  - Access rights to CPSR register itself
- Each processor mode is either
  - Privileged: full read-write access to the CPSR
  - Non-privileged: only read access to the control field of the CPSR but read-write access to the condition flags

Processor modes contd..

- ARM has seven modes
  - Privileged: abort, fast interrupt request, interrupt request, supervisor, system and undefined
  - Non-privileged: user

- User mode is used for program and applications

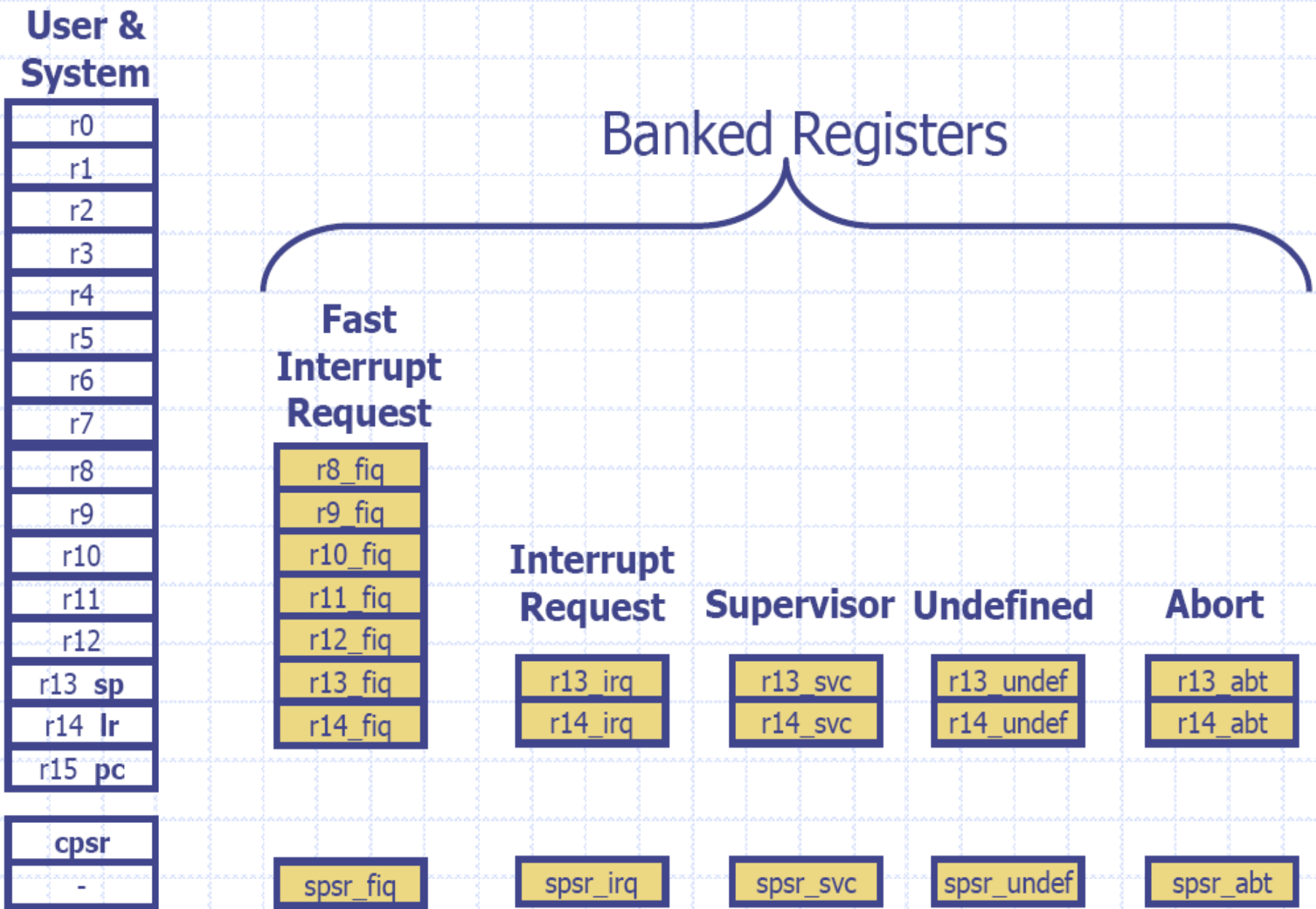
Privileged modes

- Abort: when there is a failed attempt to access memory
- Fast Interrupt Request (FIQ) & interrupt request: correspond to interrupt levels available on ARM
- Supervisor mode: state after reset and generally the mode in which OS kernel executes

Privileged modes contd..

- System mode: special version of user mode that allows full read-write access of CPSR
- Undefined: When processor encounters an undefined instruction

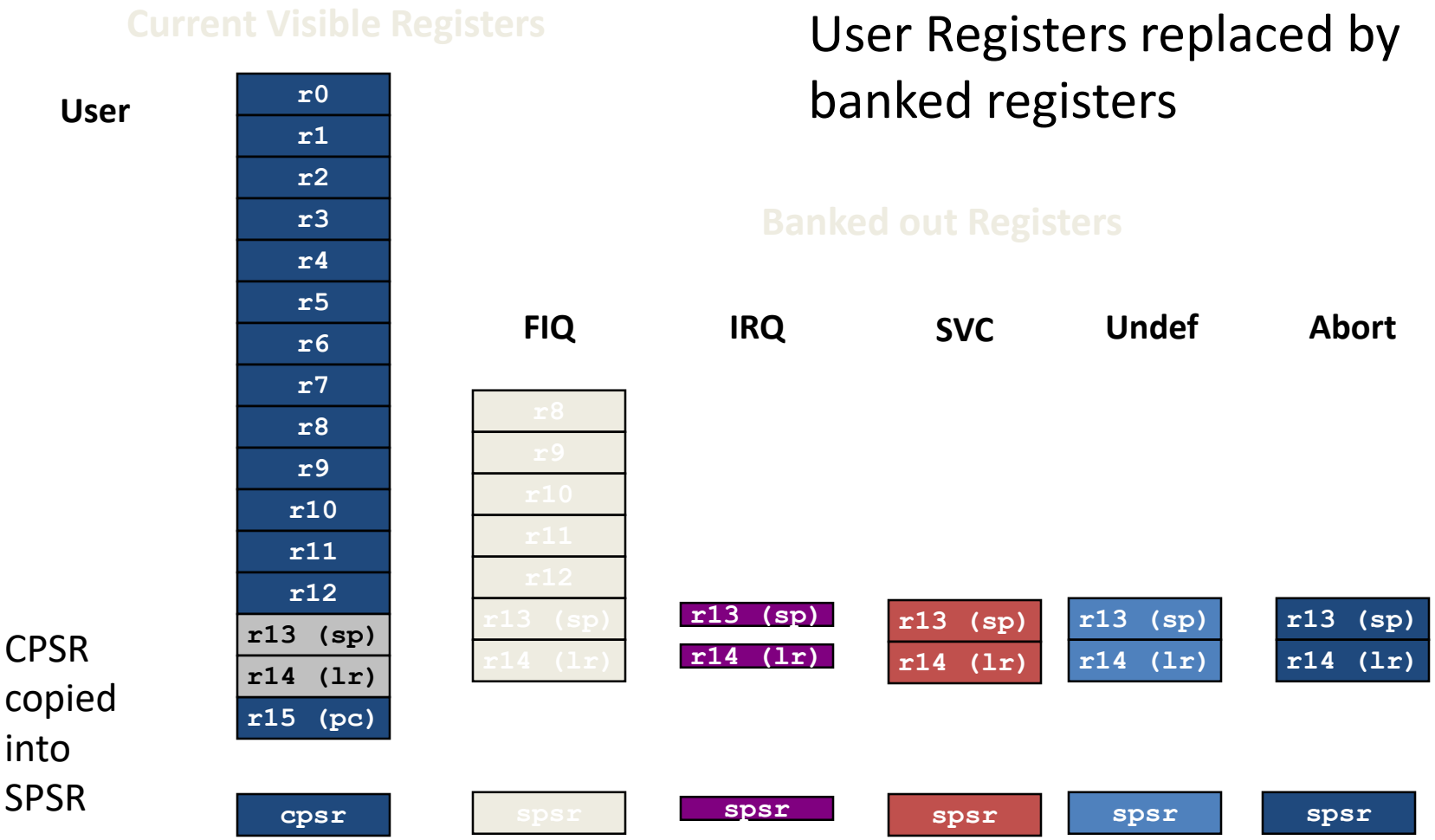
Banked Registers



Banked Registers

- Register file contains in all 37 registers
  - 20 registers are hidden from program at different times. These registers are called ***banked registers***
  - Banked registers are available only when the processor is in a particular mode
    - Processor modes (other than system mode) have a set of associated banked registers that are subset of 16 registers
    - Maps one-to-one onto a user mode register

Register Banking



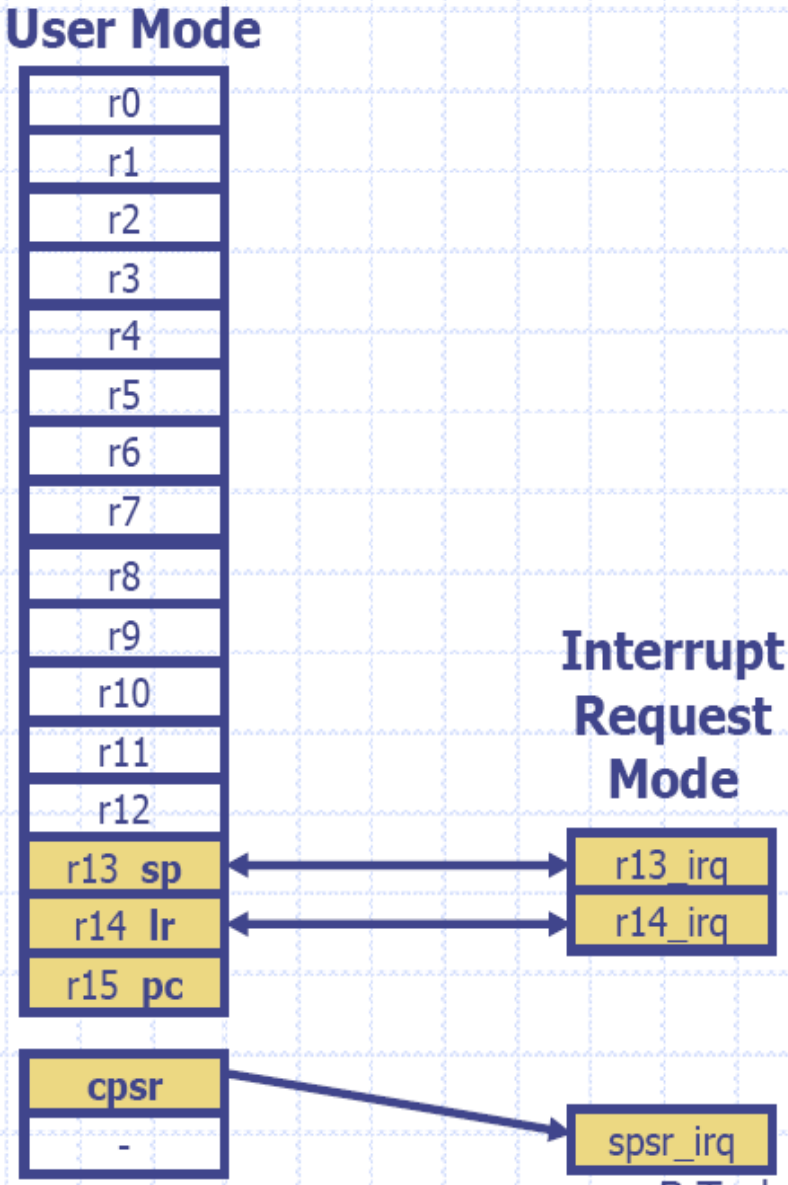
SPSR

- Each privileged mode (except system mode) has associated with it a SPSR (Stored Program Status Register)
- This SPSR is used to save the state of CPSR (Current Program Status Register) when the privileged mode is entered in order that the user state can be fully restored when the user process is resumed

Mode changing

- Mode changes by writing directly to CPSR or by hardware when the processor responds to exception (any condition that needs to halt the normal sequential execution of instructions) or interrupt
- To return to user mode a special return instruction is used that instructs the core to restore the original CPSR and banked registers

Changing mode on an exception



- This change causes user register r13 and r14 to be banked
- The user registers are replaced with registers r13_irq and r14_irq
- spsr stores the previous mode cpsr

Processor Mode

Mode	Abbr:	Privileged	Mode[4:0]
Abort	abt	yes	10111
Fast Interrupt Request	fiq	yes	10001
Interrupt Request	irq	yes	10010
Supervisor	svc	yes	10011
System	sys	yes	11111
Undefined	und	yes	11011
User	usr	no	10000

cpsr is not copied into the ***spsr*** when a mode change is forced due to a program writing directly to the ***cpsr***.

State and Instruction Sets

- There are three instruction sets
  - ARM
  - Thumb
  - Jazelle

The Jazelle instruction set is a closed instruction set and is not openly available.

To take advantage of Jazelle extra software has to be licensed from both ARM Limited and Sun Microsystems.

State and Instruction Sets

	ARM (cpsr T = 0)	Thumb (cpsr T = 1)
Instruction Size	32 bit	16 bit
Core Instruction	58	30
Conditional Execution	Most	Only branch instructions
Data Processing Instructions	Access to barrel shifter and ALU	Separate barrel and ALU instructions
Program Status Register	R/W in privileged mode	No direct access
Register Usage	15 GPR + PC	8 GPR + 7 high registers + PC

State and Instruction Sets

	Jazelle (cpsr T = 0, J = 1)
Instruction Size	8 bit
Core Instruction	Over 60% of the java bytecodes are implemented in hardware; the rest of the codes are implemented in software

Interrupt Masks

- Are used to stop specific interrupt requests from interrupting the processor
  - IRQ
  - FIQ
- The I bit masks IRQ when set to binary 1, and F bit masks FIQ when set to binary 1

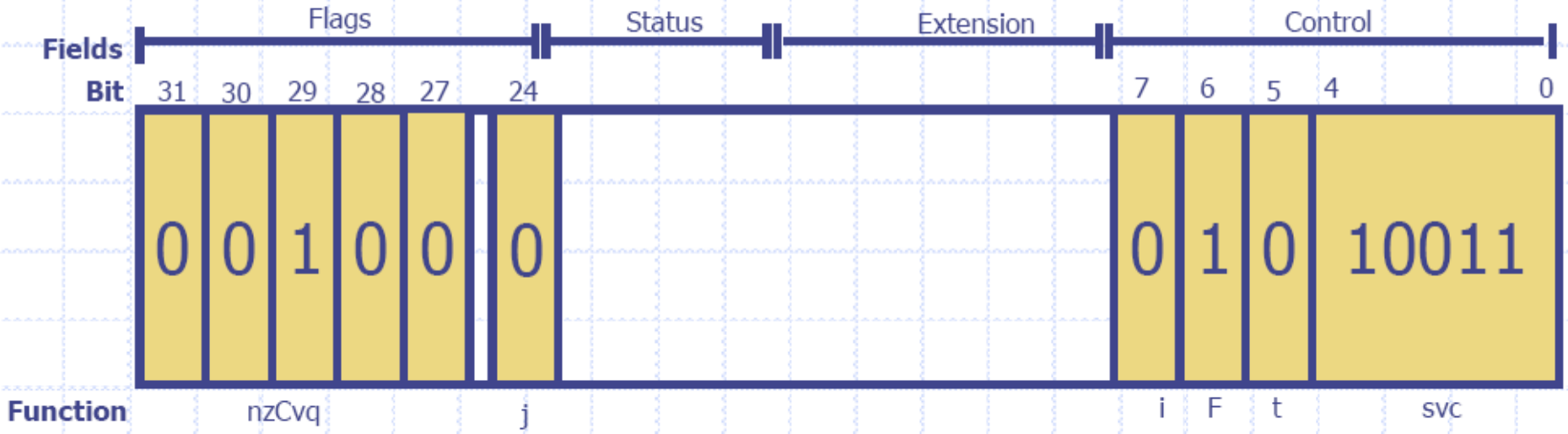
Condition Flags

Flag	Flag Name	Set when
Q	Saturation	The result causes an overflow and / or saturation
V	oVerflow	The result causes a signed overflow
C	Carry	The result causes an unsigned carry
Z	Zero	The result is zero, frequently used to indicate the equality
N	Negative	Bit 31 of the result is a binary 1

Condition Flags

- Condition flags are updated by comparisons and the result of ALU operations that specify the S instruction suffix
  - If SUBS results in a register value of zero, then the Z flag in the cpsr is set

Condition Flags – Eg



cpsr = nzCvqjiFt_SVC

Conditional Execution

- A beneficial feature of the ARM architecture is that instructions can be made to execute conditionally.
- The condition is specified with a **two-letter suffix, such as EQ or CC, appended to the mnemonic.**
- The condition is tested against the current processor flags and if not met the instruction is treated as a no-op.
- This feature often removes the need to branch and increasing speed.
- It can also increase code density.

Conditional Execution

Mnemonic	Name	Condition Flags
EQ	equal	Z
NE	not equal	z
CS HS	carry set/unsigned higher or same	C
CC LO	carry clear/unsigned lower	c
MI	minus/negative	N
PL	plus/positive or zero	n
VS	overflow	V
VC	no overflow	v
HI	unsigned higher	zC
LS	unsigned lower or same	Z or c
GE	signed greater than or equal	NV or nv
LT	signed less than	Nv or nV
GT	signed greater than	NzV or nzv
LE	signed less than or equal	Z or Nv or nV
AL	always (unconditional)	ignored

Example

- An example: `if (r2 != 10) r5 = r5 +10 - r3`

```
      CMP     r2,#10
      BEQ     SKIP
      ADD     r5,r5,r2
      SUB     r5,r5,r3
SKIP  ...
```

```
CMP       r2,#10
ADDNE     r5,r5,r2
SUBNE     r5,r5,r3
```

Example

```
if ((r1 == r3) && (r5 == r6)) r7 = r7 + 10
```

```
      CMP     r1,r3
      BNE     SKIP
      CMP     r5,r6
      BNE     SKIP
      ADD     r7,r7,#10
SKIP  ...
```

```
CMP     r1,r3
CMPEQ   r5,r6
ADDEQ   r7,r7,#10
```

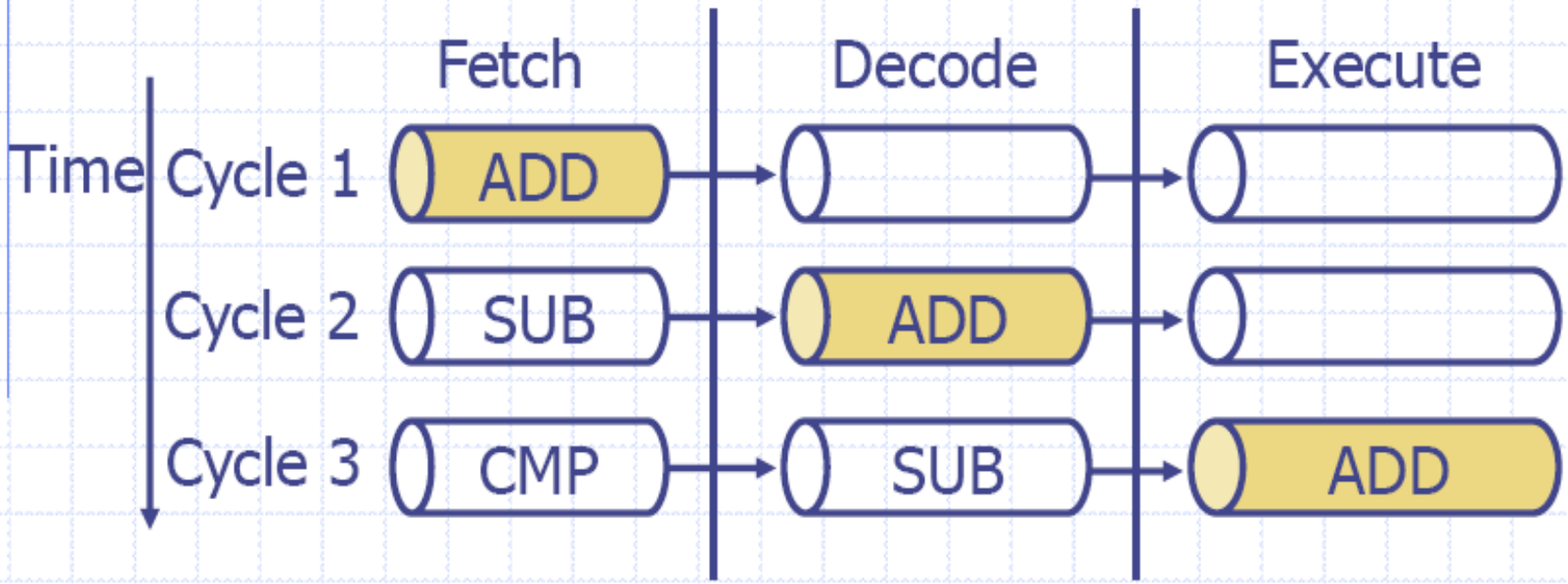
Pipeline

- Is a mechanism a RISC processor uses to execute instructions
- Using a pipeline speeds up execution by fetching the next instruction while other instructions are being decoded and executed

ARM7 Three stage pipeline

- Fetch loads an instruction from memory
- Decode identifies the instruction to be executed
- Execute processes the instruction and writes the result back to a register

Pipelined instruction sequence



- Filling the pipeline
- Allows the core to execute an instruction every cycle

ARM9 Five stage pipeline

- Fetch
  - The instruction is fetched from memory and placed in the instruction pipeline
- Decode
  - The instruction is decoded and register operands read from the register file
- Execute
  - An operand is shifted and the ALU result generated
- Memory (Buffer/Data)
  - Data memory is accessed if required. Otherwise the ALU result is buffered for one clock cycle to give the same pipeline flow for all instructions
- Write (Write-Back)
  - The results generated by the instruction are written back to the register file, including any data loaded from memory

Pipeline Characteristics

- An instruction in the execute stage will complete even though an interrupt has been raised
- The execution of a branch instruction or branching by the direct modification of the PC causes the ARM core to flush its pipeline

Exceptions, Interrupts, and the Vector Table

- When an exception or interrupt occurs, the processor set the PC to a specific memory address
- The address is within a special address range called the vector table
- The entries in the vector table are instructions that branch to specific routines designed to handle a particular exception or interrupt
- When an exception or interrupt occurs,the processor suspends normal execution and starts loading instructions from the exception vector table.

Exceptions, Interrupts, and the Vector Table

Exception / Interrupt	Shorthand	Address	High Address
Reset	RESET	0x00000000	0xffff0000
Undefined Instruction	UNDEF	0x00000004	0xffff0004
Software Interrupt	SWI	0x00000008	0xffff0008
Prefetch Abort	PABT	0x0000000C	0xffff000C
Data Abort	DABT	0x00000010	0xffff0010
Reserved	-	0x00000014	0xffff0014
Interrupt Request	IRQ	0x00000018	0xffff0018
Fast Interrupt Request	FIQ	0x0000001C	0xffff001C

Exceptions, Interrupts, and the Vector Table

- **RESET** – when power is applied, branches to initialization code
- **UNDEF** – when the processor cannot decode an instruction
- **SWI** – when a SWI instruction is called
- **PABT** – attempts to fetch an instruction from an address without the correct access permissions
- **DABT** –attempts to access data memory without the correct access permissions
- **IRQ** – by external hardware
- **FIQ** – by external hardware requiring faster response time

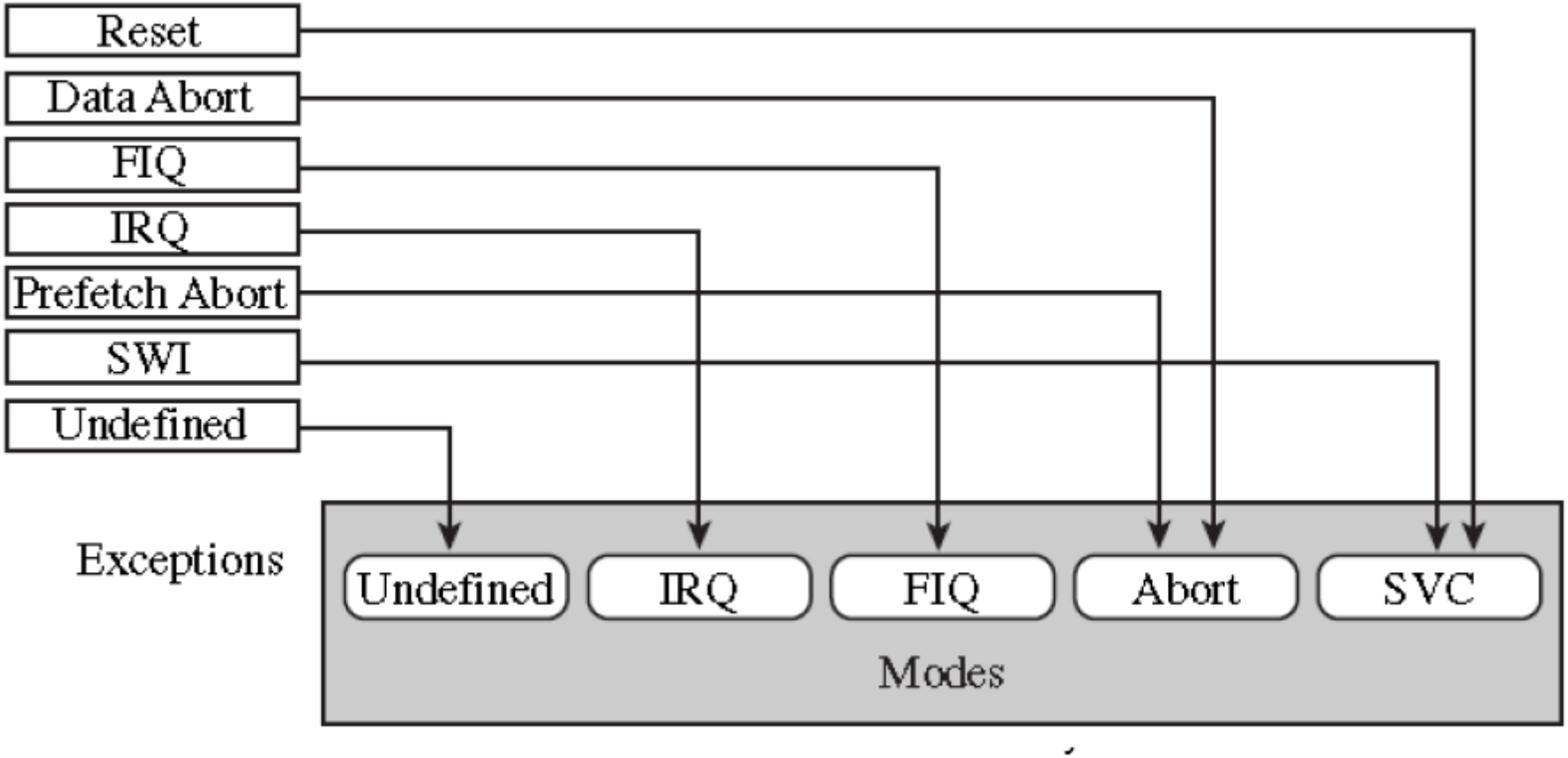
Terminology

- The terms exception and interrupt are often confused
- Exception usually refers to an internal CPU event such as
  - floating point overflow
  - MMU fault (e.g., page fault)
  - trap (SWI)
- Interrupt usually refers to an external I/O event such as
  - I/O device request
  - reset
- In the ARM architecture manuals, the two terms are mixed together

priorities

Exceptions	Priority	*I* bit	*F* bit
Reset	1	1	1
Data Abort	2	1	—
Fast Interrupt Request	3	1	1
Interrupt Request	4	1	—
Prefetch Abort	5	1	—
Software Interrupt	6	1	—
Undefined Instruction	6	1	—

ARM processor modes and Exceptions



Exception Handling

- When an exception occurs ARM processor always switches to ARM STATE.
- When an exception causes the mode change, the core automatically
  - Save CPSR to SPSR
  - PC to LR
  - Sets cpsr to exception mode
  - Set PC to address of exception handler

Here are the steps that the ARM processor does to handle an exception [5]:

- Preserve the address of the next instruction.
- Copy ***CPSR*** to the appropriate ***SPSR***, which is one of the banked registers for each mode of operation.
- Force the ***CPSR*** mode bits to a value depending on the raised exception.
- Force the ***PC*** to fetch the next instruction from the exception vector table.
- Now the handler is running in the mode associated with the raised exception.
- When handler is done, the ***CPSR*** is restored from the saved ***SPSR.***
- ***PC*** is updated with the value of (***LR*** – offset) and the offset value depends on the type of the exception.

And when deciding to leave the exception handler, the following steps occurs:

- Move the Link Register ***LR*** (minus an offset) to the ***PC***.
- Copy ***SPSR*** back to ***CPSR,*** this will automatically changes the mode back to the previous one.
- Clear the interrupt disable flags (if they were set).

ARM Memory organisation

- Can be configured as little endian or big endian
- Little endian
- Big endian

Memory organization contd..

- "Little Endian" means that the low-order byte of the number is stored in memory at the lowest address, and the high-order byte at the highest address. (The little end comes first.) For example, a 4 byte Long Int Byte3 Byte2 Byte1 Byte0 will be arranged in memory as follows:
  - Base Address+0 Byte0
  - Base Address+1 Byte1
  - Base Address+2 Byte2
  - Base Address+3 Byte3

- Intel processors (those used in PC's) use "Little Endian" byte order.

Memory Organization contd..

- "Big Endian" means that the high-order byte of the number is stored in memory at the lowest address, and the low-order byte at the highest address. (The big end comes first.) Our Long Int, would then be stored as:
  - Base Address+0 Byte3
  - Base Address+1 Byte2
  - Base Address+2 Byte1
  - Base Address+3 Byte0
- Motorola processors (those used in Mac's) use "Big Endian" byte order.